15.C57 Optimization Methods HW2

Chris Low

Problem 1

(a)

True.

We prove the statement by contrapositive. Let v^* be the optimal primal cost. Suppose

$$v^* \geq c^\top x^*.$$

Since x^* is feasible and we are minimizing,

$$v^* \leq c^\top x^*.$$

Therefore,

$$v^* = c^\top x^*,$$

so x^* is optimal.

Since x^* may be degenerate, there may be multiple bases corresponding to the same x^*, and some of these bases may not have nonnegative reduced costs. However, because x^* is already optimal, we can pivot through these different bases until we reach one with nonnegative reduced costs. If we pivot from a basis with a negative reduced cost, the pivot cannot actually improve the objective since x^* is already optimal. Therefore, these pivots must be degenerate. That is, the basis changes, but the solution remains x^*. Thus, we can reach a basis B corresponding to x^* such that

$$\bar{c}_j = c_j - c_B^\top B^{-1} A_j \geq 0 \qquad \forall j.$$

For this basis, the associated dual basic solution is

$$p^\top = c_B^\top B^{-1}.$$

Therefore,

$$c_j - p^\top A_j \geq 0 \qquad \forall j,$$

which gives

$$p^\top A_j \leq c_j \qquad \forall j.$$

Hence p is dual feasible.

Thus, if x^* is optimal, at least one basis corresponding to x^* has a feasible associated dual basic solution. By contrapositive, if every basis corresponding to x^* has an infeasible associated dual basic solution, then x^* is not optimal. Since x^* is feasible and we are minimizing,

$$v^* < c^\top x^*.$$

Therefore, the statement is true. □

(b)

True.

The original dual problem is

$$\max\ p^\top b$$

subject to

$$p^\top A \le c^\top.$$

If we remove the ith equality constraint from the primal, then the corresponding dual variable p_i is also removed. The resulting dual can be written as

$$\max \sum_{j \neq i} p_j b_j$$

subject to

$$\sum_{j \neq i} p_j A_j \le c^\top,$$

where A_j denotes the corresponding row of A.

Now consider the original dual and instead set

$$p_i = 0.$$

For the objective,

$$p^\top b = \sum_{j=1}^{m} p_j b_j = \sum_{j \neq i} p_j b_j + p_i b_i = \sum_{j \neq i} p_j b_j.$$

Similarly, for the dual constraint,

$$p^\top A = \sum_{j=1}^{m} p_j A_j = \sum_{j \neq i} p_j A_j + p_i A_i = \sum_{j \neq i} p_j A_j.$$

Therefore, setting $p_i = 0$ gives us the same objective and constraints

$$\max \sum_{j \neq i} p_j b_j$$

subject to

$$\sum_{j \neq i} p_j A_j \le c^\top,$$

as we get from removing the ith primal equality constraint. Therefore, the statement is true. □

(c)

True.

Suppose the unboundedness criterion in the primal simplex algorithm is satisfied. Then there is some nonbasic variable x_j such that

$$\bar{c}_j < 0$$

and

$$B^{-1} A_j \le 0.$$

If x_j enters the basis, the corresponding direction for the basic variables is

$$d_B = -B^{-1} A_j.$$

which implies

$$d_B \ge 0.$$

Starting from the current basic feasible solution, we can move in this direction by an amount $\theta \geq 0$:

$$x_B(\theta) = x_B + \theta d_B = x_B - \theta B^{-1} A_j.$$

Since $x_B \geq 0$ and $d_B \geq 0$, we have

$$x_B(\theta) \geq 0$$

for every $\theta \geq 0$. Therefore, θ can be increased arbitrarily while the primal solution remains feasible.

The objective value along this direction is

$$c^\top x(\theta) = c^\top x + \theta \bar{c}_j.$$

Since $\bar{c}_j < 0$, as $\theta \to \infty$,

$$c^\top x(\theta) \to -\infty.$$

Thus, the primal problem is unbounded below.

Now suppose, for contradiction, that the dual is feasible, and we let p be a dual feasible solution. Then

$$p^\top A \leq c^\top.$$

Since $x(\theta) \geq 0$, multiplying by $x(\theta)$ gives us

$$p^\top A x(\theta) \leq c^\top x(\theta).$$

And because we know now that $x(\theta)$ is primal feasible,

$$Ax(\theta) = b,$$

so

$$p^\top b \leq c^\top x(\theta).$$

However, $p^\top b$ is fixed while $c^\top x(\theta) \to -\infty$. For sufficiently large θ, we would therefore have

$$c^\top x(\theta) < p^\top b,$$

contradicting the inequality above. Hence no dual feasible solution can exist, so the dual problem is infeasible. □

(d)

True.

If x^* is a BFS, then every variable with positive flow must be a basic variable, since all nonbasic variables are equal to zero. The columns corresponding to the basic variables form the basis matrix B so they must be linearly independent for B to be invertible.

Suppose the positive-flow arcs contain an undirected cycle. The columns corresponding to the arcs in a cycle are linearly dependent. For example, for the cycle $1 \to 2 \to 3 \to 1$,

$$A_{12} + A_{23} + A_{31} = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

More generally, if some arcs point in the opposite direction around the cycle, we can add or subtract their columns so that the entries at each node cancel, again giving a nonzero linear combination equal to zero.

Therefore, the positive-flow arcs in the cycle are linearly dependent and cannot all be part of a basis. However, since they all have positive flow, they would all have to be basic if x^* were a BFS. This would require the same columns to be both linearly independent (because they are basic) and linearly dependent (because they form a cycle), which is a contradiction. Therefore, x^* cannot be a basic feasible solution. □

(e)

False.

The basic arcs of a BFS form a spanning tree, but the arcs with strictly positive flow do not necessarily form a spanning tree. This is because a BFS can be degenerate, meaning that some basic variables can be equal to zero. Therefore, some arcs can be part of the basis but carry zero flow.

For example, consider a network with nodes $1, 2, 3$ and arcs

$$(1, 2), \qquad (2, 3),$$

with supplies

$$b_1 = 1, \qquad b_2 = -1, \qquad b_3 = 0.$$

The two arcs form a spanning tree and can therefore form a basis. The corresponding feasible flow is

$$x_{12} = 1, \qquad x_{23} = 0.$$

Thus, this is a degenerate BFS since the basic variable x_{23} is equal to zero.

However, the only arc carrying strictly positive flow is $(1, 2)$. This arc does not connect node 3, so the positive-flow arcs do not form a spanning tree. Therefore, the statement is false.

Problem 2

(a)

Introduce a variable z to represent the maximum term. The original problem can be written as the LP

$$\begin{aligned} \min_{x,z} \quad & z \\ \text{s.t.} \quad & a_i^\top x - b_i \le z, \qquad i = 1, \dots, m. \end{aligned}$$

As given, we let $p \ge 0$ satisfy

$$p^\top A = 0^\top \qquad \text{and} \qquad \sum_{i=1}^{m} p_i = 1.$$

We then multiply the ith constraint by $p_i \ge 0$ and sum them over all i which gives us

$$\sum_{i=1}^{m} p_i (a_i^\top x - b_i) \le z \sum_{i=1}^{m} p_i.$$

We can see the left-hand side becomes

$$p^\top A x - p^\top b = -p^\top b,$$

because $p^\top A = 0^\top$, while the right-hand side becomes

$$z \sum_{i=1}^{m} p_i = z.$$

Therefore,

$$-p^\top b \le z$$

for every feasible (x, z). Since the minimum value of z is v, we have

$$-p^\top b \le v.$$

(b)

From part (a), every feasible p satisfies $-p^\top b \le v$. We now show that the best such lower bound is exactly v.

From part (a), the original problem can be written as

$$\begin{aligned} \min_{x,z} \quad & z \\ \text{s.t.} \quad & a_i^\top x - b_i \le z, \qquad i = 1, \dots, m. \end{aligned}$$

We multiply the constraints by -1

$$-a_i^\top x + z \ge -b_i,$$

so in matrix form the primal is

$$\begin{aligned} \min_{x,z} \quad & z \\ \text{s.t.} \quad & -Ax + \mathbf{1}z \ge -b, \end{aligned}$$

where x and z are unrestricted.

Let $p_i \ge 0$ be the dual variable to the ith primal constraint. Since each x_j is unrestricted, its corresponding dual constraint is an equality. And because the objective coefficient of each x_j is zero (we only have z in the objective above), these constraints give

$$-A^\top p = 0,$$

or equivalently,

$$p^\top A = 0^\top.$$

The variable z is also unrestricted. Its coefficient in every primal constraint is 1, and its objective coefficient is 1. Therefore, its corresponding dual constraint is

$$\mathbf{1}^\top p = 1,$$

or

$$\sum_{i=1}^{m} p_i = 1.$$

Finally, the right-hand side of the primal constraints is $-b$, so the dual objective is

$$\max_p -p^\top b.$$

Therefore, the dual problem is

$$\begin{aligned} \max_p \quad & -p^\top b \\ \text{s.t.} \quad & p^\top A = 0^\top, \\ & \sum_{i=1}^{m} p_i = 1, \\ & p \geq 0. \end{aligned}$$

This is exactly the optimization problem given in part (b). Since the primal has finite optimal value v, strong duality implies that the dual has the same optimal value. Hence,

$$\max_p \{-p^\top b : p^\top A = 0^\top,\ \mathbf{1}^\top p = 1,\ p \geq 0\} = v.$$

□

Problem 3

(a)

The objective function

$$\sum_{t=1}^{T} \alpha^{t-1}(d_t x_t - c y_t) + \alpha^T d_{T+1} z_T$$

maximizes the total discounted profit over the planning horizon. In period t, $d_t x_t$ is the revenue from selling x_t units at price d_t, while cy_t is the transportation cost of sending y_t units to long-term storage. These are discounted by α^{t-1} since future revenues and costs are valued less than current ones. Because retrieval from storage is free, we do not include w_t in the objective. Finally, any inventory z_T remaining at the end of the horizon is valued at the salvage price d_{T+1}, and we take into the discount rate as well. This gives us the discounted terminal value $\alpha^T d_{T+1} z_T$.

The first constraint

$$x_t + y_t - w_t = b_t$$

can be rewritten as

$$x_t + y_t = b_t + w_t.$$

This is the product flow balance in period t. The available product consists of new production b_t and units retrieved from storage w_t. All available product must either be sold, x_t, or sent to long-term storage, y_t.

The second constraint

$$z_t + w_t - z_{t-1} - f y_t = 0$$

can be rewritten as

$$z_t = z_{t-1} + f y_t - w_t.$$

This is the inventory balance for long-term storage. The end-of-period inventory at t equals the inventory carried over from the previous period, plus the fraction f of the newly stored product that survives, minus the amount w_t retrieved from storage.

The condition $z_0 = 0$ means that long-term storage is empty at the beginning of the planning horizon.

Finally, $x_t, y_t, w_t, z_t \geq 0$ because the quantities sold, sent to storage, retrieved, and kept in storage cannot be negative. Since demand is unlimited, we do not have an upper bound imposed on x_t.

(b)

Let p_t and q_t be the dual variables associated with the first and second equality constraints, respectively:

$$x_t + y_t - w_t = b_t,$$

and

$$z_t + w_t - z_{t-1} - f y_t = 0.$$

Because both primal constraints are equalities, the dual variables p_t and q_t are unrestricted in sign.

The primal is a maximization problem and $x_t, y_t, w_t, z_t \geq 0$, so each primal variable gives a "greater than or equal to" constraint in the dual.

For x_t, its coefficient is 1 in the first constraint and 0 in the second constraint. Its objective coefficient is $\alpha^{t-1} d_t$. Therefore,

$$p_t \geq \alpha^{t-1} d_t, \qquad t = 1, \ldots, T.$$

For y_t, its coefficients in the first and second constraints are 1 and $-f$, respectively. Its objective coefficient is $-\alpha^{t-1} c$. Therefore,

$$p_t - f q_t \geq -\alpha^{t-1} c, \qquad t = 1, \ldots, T.$$

For w_t, its coefficients in the first and second constraints are -1 and 1, respectively. Because w_t does not appear in the objective, its objective coefficient is zero. Therefore,

$$-p_t + q_t \geq 0, \qquad t = 1, \ldots, T.$$

For z_t, where $t = 1, \ldots, T-1$, it appears with coefficient 1 in the period-t storage constraint and coefficient -1 in the period-$(t+1)$ storage constraint. And because z_t does not appear in the objective for $t < T$, we obtain

$$q_t - q_{t+1} \geq 0, \qquad t = 1, \ldots, T-1.$$

Finally, we need to take into account of the last period z_T, which appears only in the period-T storage constraint with coefficient 1, and its objective coefficient is $\alpha^T d_{T+1}$. Hence,

$$q_T \geq \alpha^T d_{T+1}.$$

The dual objective is obtained from the right-hand sides of the primal constraints. The first set of constraints has right-hand side b_t, while the second set has right-hand side zero. Thus, the dual objective is

$$\min \sum_{t=1}^{T} b_t p_t.$$

Therefore, the dual problem is

$$\begin{aligned}
\min_{p,q} \quad & \sum_{t=1}^{T} b_t p_t & \\
\text{s.t.} \quad & p_t \geq \alpha^{t-1} d_t, & t = 1, \ldots, T, \\
& p_t - f q_t \geq -\alpha^{t-1} c, & t = 1, \ldots, T, \\
& -p_t + q_t \geq 0, & t = 1, \ldots, T, \\
& q_t - q_{t+1} \geq 0, & t = 1, \ldots, T-1, \\
& q_T \geq \alpha^T d_{T+1}, & \\
& p_t, q_t \text{ unrestricted in sign}, & t = 1, \ldots, T.
\end{aligned}$$

(c)

We first show that the proposed solution is dual feasible. Recall that the dual constraints can be written as

$$\begin{gathered}
p_t \geq \alpha^{t-1} d_t, \\
p_t \geq f q_t - \alpha^{t-1} c, \\
q_t \geq p_t, \\
q_t \geq q_{t+1}, \qquad t = 1, \ldots, T-1, \\
q_T \geq \alpha^T d_{T+1}.
\end{gathered}$$

By the proposed definition,

$$p_t = \max\left\{ f q_t - \alpha^{t-1} c,\, \alpha^{t-1} d_t \right\}.$$

Since a maximum is at least each of its terms, we immediately have

$$p_t \geq f q_t - \alpha^{t-1} c$$

and

$$p_t \geq \alpha^{t-1} d_t.$$

Thus, the first two dual constraints are satisfied.

For $t = 1, \ldots, T-1$, the proposed definition is

$$q_t = \max\left\{q_{t+1},\, \alpha^{t-1} d_t\right\}.$$

Therefore,

$$q_t \geq q_{t+1}$$

and

$$q_t \geq \alpha^{t-1} d_t.$$

At the final period,

$$q_T = \max\left\{\alpha^T d_{T+1},\, \alpha^{T-1} d_T\right\},$$

so

$$q_T \geq \alpha^T d_{T+1}$$

and

$$q_T \geq \alpha^{T-1} d_T.$$

Thus, the constraints involving $q_t \geq q_{t+1}$ and $q_T \geq \alpha^T d_{T+1}$ are satisfied.

It remains only to verify

$$q_t \geq p_t.$$

From the definitions of q_t and q_T, we have for every $t = 1, \ldots, T$,

$$q_t \geq \alpha^{t-1} d_t.$$

Also, since $0 < f < 1$, $c \geq 0$, and $q_t \geq 0$, we have

$$f q_t - \alpha^{t-1} c \leq f q_t \leq q_t.$$

Therefore, q_t is at least both terms in

$$p_t = \max\left\{f q_t - \alpha^{t-1} c,\, \alpha^{t-1} d_t\right\}.$$

So we conclude that

$$q_t \geq p_t.$$

Thus, all of the dual constraints are satisfied, so the proposed solution is **dual feasible**.

We now show that these choices are **optimal** by working backward from period T. Any feasible solution must satisfy (as given)

$$q_T \geq \alpha^T d_{T+1}.$$

Also, since we have

$$q_T \geq p_T \qquad \text{and} \qquad p_T \geq \alpha^{T-1} d_T,$$

we must also have

$$q_T \geq \alpha^{T-1} d_T.$$

Therefore every feasible solution must satisfy

$$q_T \geq \max\left\{\alpha^T d_{T+1},\, \alpha^{T-1} d_T\right\}.$$

Thus the proposed value $q_T = \max\left\{\alpha^T d_{T+1},\, \alpha^{T-1} d_T\right\}$ is the smallest feasible value of q_T.

Once q_T is chosen, feasibility requires

$$p_T \geq f q_T - \alpha^{T-1} c$$

and

$$p_T \geq \alpha^{T-1} d_T.$$

Hence the smallest feasible value is

$$p_T = \max\left\{fq_T - \alpha^{T-1}c,\, \alpha^{T-1}d_T\right\}.$$

We can now repeat the same argument backward. For any $t = 1, \ldots, T-1$, feasibility requires

$$q_t \geq q_{t+1}$$

and, since

$$q_t \geq p_t \geq \alpha^{t-1}d_t,$$

also

$$q_t \geq \alpha^{t-1}d_t.$$

Therefore the smallest feasible value of q_t is

$$q_t = \max\left\{q_{t+1},\, \alpha^{t-1}d_t\right\}.$$

Once q_t is determined, the two lower bounds on p_t give

$$p_t = \max\left\{fq_t - \alpha^{t-1}c,\, \alpha^{t-1}d_t\right\}$$

as the smallest feasible value of p_t.

Thus, working backward from T to 1, the proposed construction chooses the smallest feasible values of q_t and p_t in every period. Since the dual objective is $\min \sum_{t=1}^{T} b_t p_t$ and $b_t \geq 0$, choosing the smallest feasible p_t minimizes the objective. Therefore, the proposed solution is optimal. □

(d)

From part (b), the dual constraints corresponding to the primal variables are

$$p_t \geq \alpha^{t-1}d_t,$$

$$p_t - fq_t \geq -\alpha^{t-1}c,$$

$$q_t - p_t \geq 0,$$

$$q_t - q_{t+1} \geq 0, \qquad t = 1, \ldots, T-1,$$

$$q_T \geq \alpha^T d_{T+1}.$$

By complementary slackness, if a primal variable is positive, then its corresponding dual constraint must be tight. Thus,

$$x_t > 0 \quad \Rightarrow \quad p_t = \alpha^{t-1}d_t,$$

$$y_t > 0 \quad \Rightarrow \quad p_t = fq_t - \alpha^{t-1}c,$$

$$w_t > 0 \quad \Rightarrow \quad q_t = p_t,$$

and for $t < T$,

$$z_t > 0 \quad \Rightarrow \quad q_t = q_{t+1},$$

and lastly we have

$$z_T > 0 \quad \Rightarrow \quad q_T = \alpha^T d_{T+1}.$$

We first compute the optimal dual variables backward using part (c):

$$q_T = \max\left\{\alpha^T d_{T+1}, \alpha^{T-1}d_T\right\},$$

$$p_T = \max\left\{fq_T - \alpha^{T-1}c, \alpha^{T-1}d_T\right\},$$

and for $t = T-1, \ldots, 1$,

$$q_t = \max\left\{q_{t+1}, \alpha^{t-1} d_t\right\},$$
$$p_t = \max\left\{f q_t - \alpha^{t-1} c, \alpha^{t-1} d_t\right\}.$$

We then reconstruct the primal solution forward from $t = 1$ to T.

First Case:

Suppose

$$\alpha^{t-1} d_t > q_{t+1}.$$

Then

$$q_t = \alpha^{t-1} d_t > q_{t+1}.$$

Therefore the dual constraint corresponding to z_t is slack, so by complementary slackness,

$$z_t = 0.$$

Since $q_t = \alpha^{t-1} d_t$, we also have

$$p_t = \alpha^{t-1} d_t = q_t,$$

so the dual constraints corresponding to x_t and w_t are tight. Under nondegeneracy, this gives

$$x_t > 0, \qquad w_t > 0,$$

while the y_t constraint is slack, so

$$y_t = 0.$$

Using the primal balance equations,

$$z_t = z_{t-1} + f y_t - w_t,$$

we obtain

$$w_t = z_{t-1},$$

and from

$$x_t + y_t - w_t = b_t,$$

we obtain

$$x_t = b_t + z_{t-1}.$$

Hence in this case,

$$\boxed{w_t = z_{t-1}, \qquad y_t = 0, \qquad x_t = b_t + z_{t-1}, \qquad z_t = 0.}$$

It essentially means we retrieve and sell everything.

Second Case:

Suppose instead

$$q_{t+1} > \alpha^{t-1} d_t.$$

Then

$$q_t = q_{t+1},$$

so the dual constraint corresponding to z_t is tight and therefore, under nondegeneracy,

$$z_t > 0.$$

Also, $q_t > p_t$, so the dual constraint corresponding to w_t is slack, which implies

$$w_t = 0.$$

There are now two subcases.

First Subcase:

If

$$fq_t - \alpha^{t-1}c > \alpha^{t-1}d_t,$$

then

$$p_t = fq_t - \alpha^{t-1}c.$$

Thus the dual constraint corresponding to y_t is tight, while the one corresponding to x_t is slack. Hence

$$y_t > 0, \qquad x_t = 0.$$

Since $w_t = 0$, the first primal balance ($x_t + y_t - w_t = b_t$) gives

$$y_t = b_t,$$

and $z_t + w_t - z_{t-1} - fy_t = 0$ gives

$$z_t = z_{t-1} + fb_t.$$

Thus

$$\boxed{w_t = 0, \qquad y_t = b_t, \qquad x_t = 0, \qquad z_t = z_{t-1} + fb_t.}$$

It means we are storing production.

Second Subcase:

If

$$\alpha^{t-1}d_t > fq_t - \alpha^{t-1}c,$$

then

$$p_t = \alpha^{t-1}d_t.$$

Therefore the dual constraint corresponding to x_t is tight and the one corresponding to y_t is slack. Hence

$$x_t > 0, \qquad y_t = 0.$$

Since $w_t = 0$, the first primal balance constraint gives us

$$x_t = b_t,$$

and the storage balance gives

$$z_t = z_{t-1}.$$

Thus

$$\boxed{w_t = 0, \qquad y_t = 0, \qquad x_t = b_t, \qquad z_t = z_{t-1}.}$$

Summary of the algorithm.

Therefore, the back-propagation algorithm proceeds in two steps.

First, starting at $t = T$, compute q_T and p_T using the terminal formulas from part (c). Then move backward from $t = T - 1$ to $t = 1$, computing q_t and p_t recursively. At each period, record which of the three cases above applies.

After completing this backward pass, set $z_0 = 0$.

Then move forward from $t = 1$ to $t = T$. In each period, use the case identified during the backward pass together with the primal balance equations to determine x_t, y_t, w_t, and z_t.

Thus, the backward pass determines the optimal action in each period, while the forward pass determines the corresponding quantities. By complementary slackness, the resulting primal solution is optimal.

Problem 4.

(a)

Let $x_{ij} \geq 0$ denote the fraction of job i processed on machine j. The linear optimization problem is

$$\begin{aligned}
\min \quad & \sum_{i=1}^{1000} \sum_{j=1}^{20} c_{ij} x_{ij} \\
\text{s.t.} \quad & \sum_{j=1}^{20} x_{ij} = 1, \qquad i = 1, \ldots, 1000, \\
& \sum_{i=1}^{1000} x_{ij} \leq C_j, \quad j = 1, \ldots, 20, \\
& x_{ij} \geq 0, \qquad\quad i = 1, \ldots, 1000, \quad j = 1, \ldots, 20.
\end{aligned}$$

The first constraint ensures that each job is fully processed. The second constraint ensures that machine j processes at most C_j units of jobs.

This problem can be represented as a minimum-cost network flow problem. We can create one node for each job, one node for each machine, and one sink node s. Each job node i has supply 1, each machine node has supply/demand 0, and the sink has demand 1000. For every job i and machine j, we can create an arc from job i to machine j with cost c_{ij} and capacity 1. The flow on this arc is x_{ij}. For every machine j, we can create an arc from machine j to the sink with cost 0 and capacity C_j. Thus the network has the form

$$\boxed{\text{Job } i} \xrightarrow[\text{capacity } 1]{\text{cost } c_{ij}} \boxed{\text{Machine } j} \xrightarrow[\text{capacity } C_j]{\text{cost } 0} \boxed{\text{Sink}}$$

where the job-to-machine arc has cost c_{ij} and capacity 1, and the machine-to-sink arc has cost 0 and capacity C_j.

We need flow balance at job i which gives us $\sum_{j=1}^{20} x_{ij} = 1$. We also need flow balance at machine j, which together with the capacity of its arc to the sink gives us $\sum_{i=1}^{1000} x_{ij} \leq C_j$. Therefore, this network flow formulation is equivalent to the original linear optimization problem.

For a minimum-cost network flow problem, the integrality theorem states that if all supplies, demands, and arc capacities are integers, then there exists an optimal solution in which all flows are integers.

In our network, each job has supply 1, the sink has demand 1000, and the arc capacities are 1 and C_j. Assuming that each C_j is an integer, all of these quantities are integers. Therefore, by the integrality theorem, there exists an optimal integral flow. In particular, the flows x_{ij} are integers. Since $x_{ij} \geq 0$ and $\sum_{j=1}^{20} x_{ij} = 1$ for each job i, we must have $x_{ij} \in \{0, 1\}$. Thus, there exists an optimal solution in which each job is processed entirely on a single machine, even though the model allows jobs to be split across multiple machines.

(c)

(Note: Parts (b), (e), and (f) are attached to the end of the document.)

From part (a), the primal problem is

$$\begin{aligned}
\min \quad & \sum_{i=1}^{1000}\sum_{j=1}^{20} c_{ij}x_{ij} \\
\text{s.t.} \quad & \sum_{j=1}^{20} x_{ij} = 1, \qquad i = 1,\dots,1000, \\
& \sum_{i=1}^{1000} x_{ij} \le C_j, \quad j = 1,\dots,20, \\
& x_{ij} \ge 0, \qquad \forall i,j.
\end{aligned}$$

We let p_i be the dual variable associated with the job constraint $\sum_{j=1}^{20} x_{ij} = 1$,

and let q_j be the dual variable associated with the machine capacity constraint $\sum_{i=1}^{1000} x_{ij} \le C_j$.

Since the job constraints are equalities, p_i is free in sign. Since the machine capacity constraints are $\le$ constraints in a minimization problem, $q_j \le 0$.

The right-hand sides of the primal constraints are 1 and C_j, so the dual objective is

$$\max \left\{ \sum_{i=1}^{1000} p_i + \sum_{j=1}^{20} C_j q_j \right\}.$$

For the primal variable, each of x_{ij} appears with coefficient 1 in the job i constraint and coefficient 1 in the machine j constraint. Since $x_{ij} \ge 0$, the corresponding dual constraint is $p_i + q_j \le c_{ij}$.

Therefore, the dual problem is

$$\begin{aligned}
\max \quad & \sum_{i=1}^{1000} p_i + \sum_{j=1}^{20} C_j q_j \\
\text{s.t.} \quad & p_i + q_j \le c_{ij}, && i = 1,\dots,1000, && j = 1,\dots,20, \\
& p_i \text{ free}, && i = 1,\dots,1000, \\
& q_j \le 0, && j = 1,\dots,20.
\end{aligned}$$

Interpretation of the dual variables. The dual variable p_i is associated with the constraint that job i must be fully processed. It can be interpreted as the marginal energy cost of increasing the amount of job i that needs to be processed.

The dual variable q_j is associated with the capacity of machine j. Since $q_j \le 0$, a negative value means that increasing the capacity of machine j would decrease the minimum total energy consumption. If $q_j = 0$, then increasing the capacity of that machine would not improve the optimal objective value.

The dual constraint $p_i + q_j \le c_{ij}$ can also be written as $p_i \le c_{ij} - q_j$. Thus, $c_{ij} - q_j$ can be viewed as the energy cost of assigning job i to machine j, adjusted for how valuable the capacity of machine j is. Since the dual maximizes p_i, p_i will be limited by the machine with the smallest adjusted cost.

Complementary slackness. For each x_{ij}, complementary slackness gives

$$x_{ij}\,(c_{ij} - p_i - q_j) = 0.$$

Therefore, if job i is assigned to machine j, so that $x_{ij} > 0$, then $p_i + q_j = c_{ij}$. Equivalently, $p_i = c_{ij} - q_j$.

Since the dual constraints require $p_i \leq c_{ij} - q_j$ for every machine j, and the dual maximizes p_i, we have $p_i = \min_j\{c_{ij} - q_j\}$. Therefore, job i can only be assigned to a machine that minimizes $c_{ij} - q_j$. If $p_i + q_j < c_{ij}$, then $x_{ij} = 0$, so job i is not assigned to machine j.

For the machine capacity constraints, complementary slackness gives

$$q_j \left(C_j - \sum_{i=1}^{1000} x_{ij} \right) = 0.$$

Thus, if $q_j < 0$, then $\sum_{i=1}^{1000} x_{ij} = C_j$, so machine j is operating at full capacity. Conversely, if $\sum_{i=1}^{1000} x_{ij} < C_j$, then $q_j = 0$. Therefore, a machine with unused capacity has zero marginal value for additional capacity.

(d)

Let $x_{ij} \geq 0$ be the fraction of job i processed on machine j. Since processing all of job i on machine j uses r_{ij} units of utilization, processing a fraction x_{ij} uses $r_{ij}x_{ij}$ units.

$$\begin{aligned}
\min \quad & \sum_{i=1}^{1000} \sum_{j=1}^{20} c_{ij} x_{ij} \\
\text{s.t.} \quad & \sum_{j=1}^{20} x_{ij} = 1, && i = 1, \ldots, 1000, \\
& \sum_{i=1}^{1000} r_{ij} x_{ij} \leq U_j, && j = 1, \ldots, 20, \\
& x_{ij} \geq 0, && \forall i, j.
\end{aligned}$$

This is not a standard network flow problem. In part (a), each job used the same amount of machine capacity, so the amount of flow into a machine directly represented the amount of capacity being used. Here, each job can use a different amount of a machine's utilization because of r_{ij}. Therefore, we cannot represent the utilization constraint using a normal capacity constraint on a network arc.

The solution is also not guaranteed to be integral. For example, suppose there is one job and two machines, where

$$c_{11} = 0, \qquad c_{12} = 1,$$

$$r_{11} = 2, \qquad r_{12} = 1,$$

and

$$U_1 = U_2 = 1.$$

Then

$$x_{11} + x_{12} = 1$$

and the utilization constraint for machine 1 is

$$2x_{11} \leq 1.$$

Since machine 1 has the lower cost, we would want to assign as much of the job to machine 1 as possible. This gives

$$x_{11} = \frac{1}{2}, \qquad x_{12} = \frac{1}{2}.$$

Thus, the optimal solution can split a job between machines. Therefore, unlike in part (a), we are not guaranteed to have an integral optimal solution where each job is assigned to only one machine.

In [1]:
```
using CSV, DataFrames, JuMP, Gurobi, Plots, Printf, Statistics
gr(); default(left_margin = 6Plots.mm, bottom_margin = 6Plots.mm)
```

Data

In [ ]:
```
datadir = joinpath(@__DIR__, "HW2_data")
readmat(f) = Matrix{Float64}(CSV.read(joinpath(datadir, f), DataFrame)[:, 2:end])

c = readmat("energy.csv")
r = readmat("utilization.csv")
C = CSV.read(joinpath(datadir, "capacity.csv"), DataFrame).capacity
U = CSV.read(joinpath(datadir, "maxutil.csv"),  DataFrame).maxutil

n, m = size(c)
@assert size(r) == (n, m) && length(C) == m && length(U) == m
(n = n, m = m, total_capacity = sum(C), total_maxutil = sum(U))
```

Out[ ]:
```
(n = 1000, m = 20, total_capacity = 1000.0, total_maxutil = 30045.508167750442)
```

In [ ]:
```
function solve_allocation(w, K)
    model = Model(Gurobi.Optimizer)
    set_silent(model)

    @variable(model, x[1:n, 1:m] >= 0)
    @constraint(model, assign[i = 1:n], sum(x[i, j] for j in 1:m) == 1)
    @constraint(model, machine[j = 1:m], sum(w[i, j] * x[i, j] for i in 1:n) <= K[j])
    @objective(model, Min, sum(c[i, j] * x[i, j] for i in 1:n, j in 1:m))

    optimize!(model)
    @assert termination_status(model) == MOI.OPTIMAL
    return (obj = objective_value(model), X = value.(x), q = dual.(machine), p = dual.(assign))
end

tol = 1e-6
best = [argmin(c[i, :]) for i in 1:n];
```

In [4]:
```
E_free = sum(minimum(c[i, :]) for i in 1:n)
@printf("Energy with no machine constraints: %.2f\n", E_free)
```

```
Energy with no machine constraints: 40883.16
```

(b)

In [5]:
```
cap = solve_allocation(ones(n, m), C)

not_best_b   = count(i -> cap.X[i, best[i]] < 1 - tol, 1:n)
split_b      = count(i -> maximum(cap.X[i, :]) < 1 - tol, 1:n)
jobs_per_mach = vec(sum(cap.X, dims = 1))

@printf("1. Optimal energy consumption              : %.2f\n", cap.obj)
@printf("2. Energy ignoring capacities              : %.2f\n", E_free)
@printf("   Cost of capacity limits                 : %.2f (%.2f%%)\n", cap.obj - E_free, 100 * (ca
@printf("3. Jobs not fully on lowest-energy machine : %d of %d\n", not_best_b, n)
println()
println("Jobs split across machines: $split_b")
println("All machines at full capacity: $(all(jobs_per_mach .>= C .- tol))")
```

```
Set parameter Username
Set parameter LicenseID to value 2857457
Academic license - for non-commercial use only - expires 2027-08-26
1. Optimal energy consumption              : 41416.51
2. Energy ignoring capacities              : 40883.16
   Cost of capacity limits                 : 533.35 (1.30%)
3. Jobs not fully on lowest-energy machine : 386 of 1000

Jobs split across machines: 0
All machines at full capacity: true
```

In [6]:
```
DataFrame(machine = 1:m, capacity = C, jobs_assigned = round.(jobs_per_mach, digits = 3),
          jobs_preferring = [count(==(j), best) for j in 1:m])
```

Out[6]: 20×4 DataFrame

Row	machine	capacity	jobs_assigned	jobs_preferring
	Int64	Float64	Float64	Int64
1	1	23.0	23.0	54
2	2	24.0	24.0	58
3	3	30.0	30.0	42
4	4	4.0	4.0	48
5	5	70.0	70.0	47
6	6	87.0	87.0	43
7	7	75.0	75.0	43
8	8	107.0	107.0	49
9	9	43.0	43.0	50
10	10	52.0	52.0	54
11	11	4.0	4.0	55
12	12	20.0	20.0	48
13	13	49.0	49.0	49
14	14	69.0	69.0	53
15	15	141.0	141.0	49
16	16	32.0	32.0	47
17	17	35.0	35.0	49
18	18	4.0	4.0	40
19	19	23.0	23.0	69
20	20	108.0	108.0	53

The optimal energy consumption is $41{,}416.51$. If machine capacities were ignored, the minimum energy consumption would be $40{,}883.16$. Thus, the capacity constraints increase total energy consumption by 533.35, or about 1.3%.

In the optimal solution, 386 of the 1000 jobs are not assigned to their lowest-energy machine. This happens because some of the machines that are best for many jobs do not have enough capacity to process all of them, so some jobs must be assigned to their second- or third-best machines.

Also, since $\sum_{j=1}^{20} C_j = 1000$, which is exactly the total number of jobs, every machine must be used at full capacity. The solution is integral, so no job is split across multiple machines, which is consistent with the network-flow integrality result from part (a).

(e)

In [7]:
```
util = solve_allocation(r, U)

not_best_e = count(i -> util.X[i, best[i]] < 1 - tol, 1:n)
split_e    = count(i -> maximum(util.X[i, :]) < 1 - tol, 1:n)
used_util  = [sum(r[i, j] * util.X[i, j] for i in 1:n) for j in 1:m]

@printf("1. Optimal energy consumption              : %.2f\n", util.obj)
@printf("2. Energy ignoring utilization             : %.2f\n", E_free)
@printf("   Cost of utilization limits              : %.2f (%.2f%%)\n", util.obj - E_free, 100 * (u
@printf("3. Jobs not fully on lowest-energy machine : %d of %d\n", not_best_e, n)
println()
println("Jobs split across machines: $split_e")
println("Machines at full utilization: $(count(used_util .>= U .- tol)) of $m")
```

```
Set parameter Username
Set parameter LicenseID to value 2857457
Academic license - for non-commercial use only - expires 2027-08-26
1. Optimal energy consumption              : 46693.58
2. Energy ignoring utilization             : 40883.16
```

```
   Cost of utilization limits              : 5810.42 (14.21%)
3. Jobs not fully on lowest-energy machine : 885 of 1000

Jobs split across machines: 20
Machines at full utilization: 20 of 20
```

The optimal energy consumption is $46{,}693.58$. If the utilization constraints were ignored, the minimum energy consumption would be $40{,}883.16$. Thus, the utilization constraints increase total energy consumption by $5{,}810.42$, or about 14.2%.

In the optimal solution, 885 of the 1000 jobs are not fully assigned to their lowest-energy machine. This is because the utilization constraints are more restrictive than the simple capacity constraints in part (b). We have different jobs use different amounts of utilization on different machines.

Unlike part (b), the optimal solution is not integral. In this solution, 20 jobs are split across multiple machines. All 20 machines also operate at their maximum utilization.

(f)

In [8]:
```
q = util.q    # JuMP convention: q[j] ≤ 0 for a ≤ constraint in a minimization, matching part (c)
DataFrame(machine = 1:m, maxutil = round.(U, digits = 1), dual = round.(q, digits = 4),
          util_used = round.(used_util ./ U, digits = 4))
```

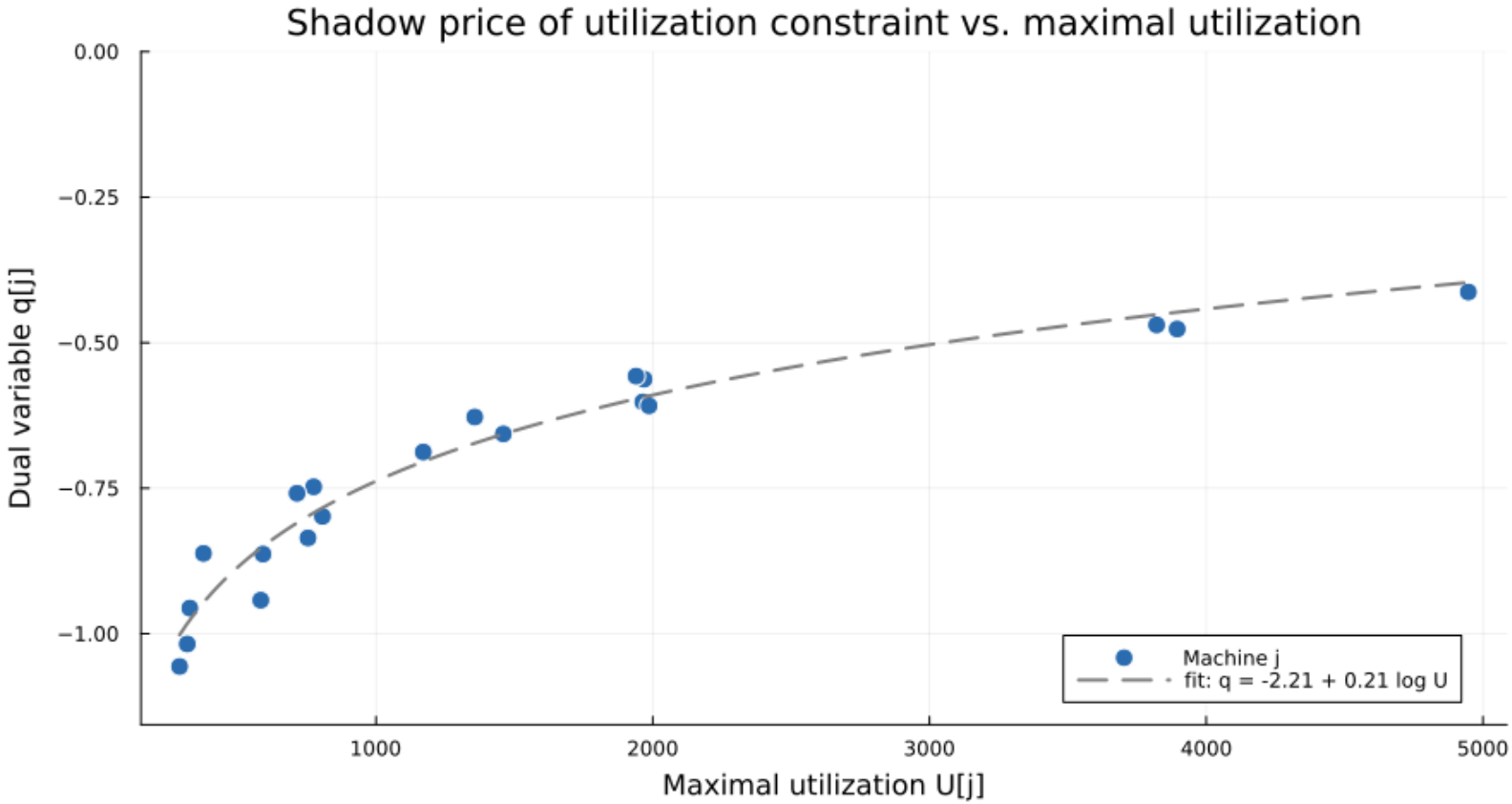
Out[8]: 20×4 DataFrame

Row	machine	maxutil	dual	util_used
	Int64	Float64	Float64	Float64
1	1	1170.7	-0.6877	1.0
2	2	1356.7	-0.6276	1.0
3	3	774.6	-0.7476	1.0
4	4	583.4	-0.9423	1.0
5	5	806.4	-0.7983	1.0
6	6	327.1	-0.9559	1.0
7	7	3895.1	-0.4764	1.0
8	8	591.4	-0.8631	1.0
9	9	1968.6	-0.5627	1.0
10	10	376.5	-0.8619	1.0
11	11	1964.1	-0.6017	1.0
12	12	754.1	-0.8353	1.0
13	13	3820.9	-0.4694	1.0
14	14	317.8	-1.0176	1.0
15	15	1939.7	-0.5574	1.0
16	16	714.5	-0.7586	1.0
17	17	1460.1	-0.6567	1.0
18	18	1986.4	-0.6083	1.0
19	19	290.1	-1.0562	1.0
20	20	4947.2	-0.4129	1.0

In [ ]:
```
a, b = [ones(m) log.(U)] \ q
Ugrid = range(minimum(U), maximum(U), length = 200)

scatter(U, q; color = "#2b6cb0", markersize = 6, markerstrokecolor = :white, label = "Machine j",
        xlabel = "Maximal utilization U[j]", ylabel = "Dual variable q[j]",
        title = "Shadow price of utilization constraint vs. maximal utilization",
        ylims = (minimum(q) - 0.1, 0), legend = :bottomright, size = (900, 480))
plot!(Ugrid, a .+ b .* log.(Ugrid); color = :gray, linestyle = :dash, linewidth = 2,
      label = @sprintf("fit: q = %.2f + %.2f log U", a, b))
```

Out[ ]:



In [10]:
```
@printf("Range of duals: [%.3f, %.3f]\n", minimum(q), maximum(q))
@printf("Correlation(U, q)      = %.3f\n", cor(U, q))
@printf("Correlation(log U, q)  = %.3f\n", cor(log.(U), q))
```

```
Range of duals: [-1.056, -0.413]
Correlation(U, q)      = 0.878
Correlation(log U, q)  = 0.976
```

Comments.

All of the dual variables are negative, ranging from approximately -1.06 to -0.41. Since each $q_j < 0$, complementary slackness implies that all of the machine utilization constraints are binding.

The magnitude of q_j represents the value of additional utilization capacity. For example, a dual value of -1.06 means that increasing the maximal utilization of that machine by one unit would reduce the optimal energy consumption by approximately 1.06 units.

From the scatter plot, machines with smaller maximal utilization tend to have more negative dual values, while machines with larger maximal utilization tend to have dual values closer to zero. Thus, additional utilization appears to be more valuable for machines with tighter utilization limits.